Vulnerability Management Pipeline

Risk-based prioritization across cloud, network, and container findings

17,148	398	33	5
findings collected	P1 (highest priority)	actively exploited (KEV)	fixes remediated & verified

The problem

Any real environment produces thousands of vulnerability findings, and no team can fix thousands of issues at once. Finding vulnerabilities is the easy part — a scanner does it automatically. The real work is deciding **what to fix first** with limited resources, driven by **real-world exploitability** rather than raw severity. This project answers one question end to end: out of 17,000+ findings, which ones actually matter right now?

Approach

I built a pipeline that mirrors how a real vulnerability-management program operates, in an isolated home lab I own (host-only network) plus my own AWS account:

Scan	Normalize	Enrich	Prioritize	Remediate	Verify	Dashboard

- **Three scan sources, three domains:** Prowler for AWS misconfigurations (cloud posture), Greenbone/OpenVAS for network hosts, and Trivy for container images. Each sees a different attack surface — together they cover how real systems are actually exposed.
- **Normalization (Python):** the three tools output three incompatible formats. A script unifies all findings into one common schema so they can be compared and ranked together.
- **Enrichment (Python):** each CVE is enriched with **CISA KEV** (is it being exploited in the wild?) and **FIRST EPSS** (how likely is it to be exploited?). This is what turns raw severity into real risk.
- **Scoring (Python):** findings are assigned a P1–P4 priority with remediation SLAs; anything in KEV is escalated to P1 regardless of its base score.

Results

The pipeline reduced **17,148 raw findings** to **398 P1 items**, and — most importantly — surfaced the **33 findings confirmed as actively exploited** in the real world. That is the difference between "17,000 problems" and "here are the ones an attacker could use today." I then closed the loop: **5 high-priority AWS misconfigurations were remediated and verified** directly against the AWS API (deleted an internet-open SSH security group, removed access keys from a non-MFA user, enabled S3 Block Public Access, default EBS encryption, and enforced IMDSv2).

Key engineering decisions

- **Prioritize by exploitability, not volume.** 99% of findings came from one outdated container image, but volume is not risk — the 33 KEV items outrank thousands of unexploited criticals.
- **Keep raw scan data out of version control.** Large, regenerable, and potentially sensitive outputs are gitignored; the repo versions the scripts and summarized evidence instead.
- **Remediate what a company actually fixes first.** Rather than chasing an impossible "fix everything," I remediated the standard CIS-Benchmark cloud misconfigurations and verified each one.

Skills demonstrated

Vulnerability management lifecycle · cloud security (AWS, CSPM) · Python data processing (normalize / enrich / score) · CVE / CVSS / KEV / EPSS risk modeling · container & network scanning (Trivy, Greenbone) · AWS CLI remediation · Power BI reporting · Git/GitHub workflow.

Full source, scripts, documentation and dashboard: **github.com/aragonofficialaam-glitch/vuln-mgmt-pipeline** · All testing was performed only against systems I own and am authorized to test.